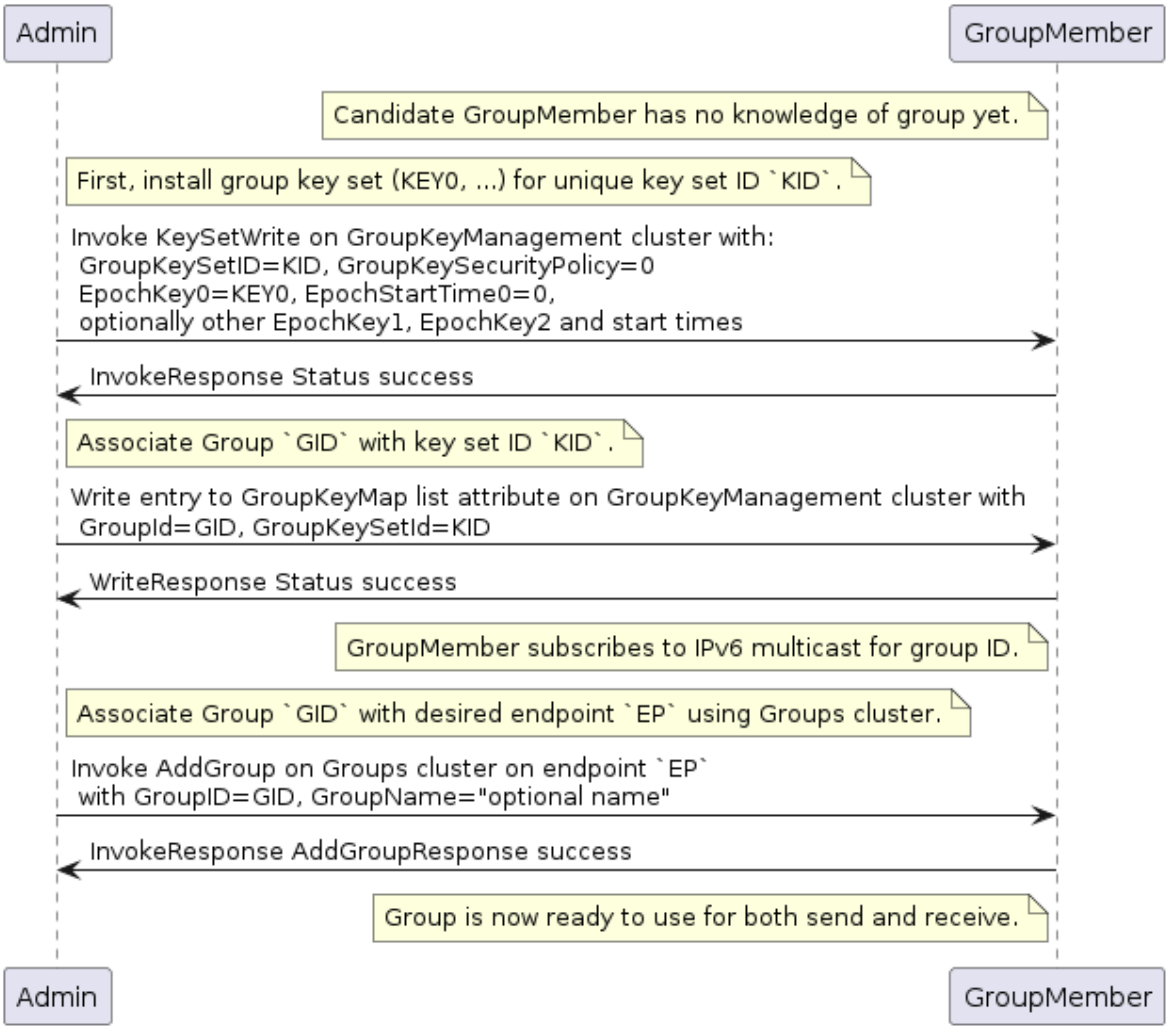

Figure 20. Installing a group onto a new node

Sequence:

- *Admin*:
  - Generate fabric-unique group ID `GID` and random key `key0` for group key set ID `K`.
  - Write the group key set `K` to the remote node, GroupMember, using `KeySetWrite` command.
  - Associate group ID `GID` with key set `K` by writing an entry to the GroupKeyMap list attribute.
- *GroupMember*:
  - Node subscribes to the IPv6 multicast address generated from the fabric ID and group ID.
- *Admin*:
  - Associate endpoint with group ID `GID` by sending the `Groups` cluster's `AddGroup` command to endpoint.

4.18. Message Counter Synchronization Protocol (MCSP)

This section describes the protocol used to securely synchronize the message counter used by a

sender of messages encrypted with a symmetric group key.

Message counter synchronization is an essential part of enabling secure messaging between members of an operational group. Specifically, it protects against replay attacks, where an attacker replays older messages, which may result in unexpected behavior if accepted and processed by the receiver.

The recipient of a message encrypted with a group key can trust and process that message only if it has kept the last message counter received from a given sender using that key.

Underlying MCSP is a simple request / response protocol. When a multicast message with unknown counter is received, synchronization via MCSP begins by sending a synchronization request via unicast UDP to the multicast message originator's unicast IPv6 address. That originator then sends a synchronization response to the unsynchronized node via unicast UDP. After cryptographic verification, the formerly unsynchronized node is now synchronized with the originator's message counter and can trust the original and subsequent messages from the originator node.

4.18.1. Message Counter Synchronization Methods

There are two methods for synchronizing the message counter of a peer node, which are configurable per-group-key based on the GroupKeySecurityPolicy field of a given group key set (see GroupKeySetStruct).

4.18.1.1. Trust-first policy

The first authenticated message counter from an unsynchronized peer is trusted, and its message counter is used to configure message-counter-based replay protection on future messages from that node. All control messages (any message with C Flag set) use the control message counter and SHALL use Trust-first for synchronization. Note that MCSP is not used for Trust-first synchronization.

This policy provides lower latency for less security-sensitive applications such as lighting.

WARNING	Trust-first synchronization is susceptible to accepting a replayed message after a Node has been rebooted.

4.18.1.2. Cache-and-sync policy

The message that triggers message counter synchronization is stored, a message counter synchronization exchange is initiated, and only when the synchronization is completed is the original message processed. Cache-and-sync provides replay protection even in the case where a Node has been rebooted, at the expense of higher latency.

Support for the cache-and-sync policy and MCSP is optional. A node indicates its ability to support this feature via the Group Key Management cluster FeatureMap.

WARNING	Large groups may cause significant message synchronization burden on the sender and message queueing pressure or latency on the receiver when message counter synchronization state is out of sync, such as after a brown-out, on

first message to a new large group, or similar large scale sets of synchronizations being required at a single point in time.

4.18.2. Group Peer State

The *Group Peer State Table* stores information about every peer with which the node had a group message exchange. For every peer node id the following information is available in the table:

- Peer's Encrypted Group Data Message Counter Status:
  - *Synchronized Data Message Counter* - the largest encrypted data message counter received from the peer, if available.
  - Flag to indicate whether this counter value is valid and synchronized.
  - The *message reception state* bitmap tracking the recent window of data message counters received from the peer.
- Peer's Encrypted Group Control Message Counter Status:
  - *Synchronized Control Message Counter* - the largest encrypted control message counter received from the peer, if available.
  - Flag to indicate whether this counter value is valid and synchronized.
  - The *message reception state* bitmap tracking the recent window of control message counters received from the peer.

There are three scenarios where the receiver of an encrypted message does not know the sender's last message counter:

- The encrypted message is the first received from a peer.
- The device rebooted without persisting the *Group Peer State Table* content.

NOTE It is not required to persist the Peer State Table.

- The entry for the Peer in the *Group Peer State Table* was expunged due to the table being full.

There SHALL be at least 10 entries per supported fabric for Peer Encrypted Group data Message Status in the Group Peer State table. This number SHOULD NOT be less than 5 entries per fabric per instance of Groups cluster on an endpoint. The number of entries is a compromise between the number of peers that can send a group message to a receiver while the table is maintained, and the associated memory usage.

There SHALL be at least 2 entries per supported fabric for Peer Encrypted Group Control Message Status in the Group Peer State table.

The next sections describe the functional protocol used to request message counter synchronization with a peer and form responses to such message counter synchronization requests.

4.18.3. MCSP Messages

4.18.3.1. MsgCounterSyncReq - Message Counter Synchronization Request

The Message Counter Synchronization Request (*MsgCounterSyncReq*) message is sent when a message was received from a peer whose current message counter is unknown.

A *MsgCounterSyncReq* message SHALL set the C Flag in the message header. The control message counter SHALL be used for message protection.

A *MsgCounterSyncReq* message SHALL be secured with the group key for which counter synchronization is requested and SHALL set the Session Type to 1, indicating a group session as per the rules outline in Section 4.18.5, “Message Counter Synchronization Exchange”.

The payload of the *MsgCounterSyncReq* message takes the format defined in Table 26, “Message Counter Sync Request”:

Table 26. Message Counter Sync Request

Field Size	Message Field	Description
8 bytes	Challenge	The Challenge is a 64-bit random number generated using the DRBG by the initiator of a *MsgCounterSyncReq* to uniquely identify the synchronization request cryptographically.

4.18.3.2. MsgCounterSyncRsp - Message Counter Synchronization Response

The Message Counter Synchronization Response (*MsgCounterSyncRsp*) message is sent in response to a *MsgCounterSyncReq*.

A *MsgCounterSyncRsp* message SHALL set the C Flag in the message header. The control message counter SHALL be used for message protection.

The *MsgCounterSyncRsp* message has the format defined in Table 27, “Message Counter Sync Response”:

Table 27. Message Counter Sync Response

Field Size	Message Field	Description
4 bytes	Synchronized Counter	The current data message counter for the node sending the *MsgCounterSyncRsp* message.
8 bytes	Response	The Response SHALL be the same as the 64-bit value sent in the Challenge field of the corresponding *MsgCounterSyncReq*.

4.18.4. Unsynchronized Message Processing

An unsynchronized message is one that is cryptographically verified from a node whose message counter is unknown. Upon receipt of an unsynchronized message process the message as follows:

1. The message SHALL be of Group Session Type, otherwise discard the message.
2. If C Flag is set to 1:
    a. Create a Message Reception State and set its `max_message_counter` to the message counter of the given message, i.e. trust-first.
    b. Accept the message for further processing.
3. If C Flag is set to 0:
    a. Determine the GroupKeySecurityPolicy (as set by the Section 11.2, “Group Key Management Cluster”) of the operational group key used to authenticate the message.
    b. If the key has a *trust-first* security policy, the receiver SHALL:
        i. Set the peer’s group key data message counter to *Message Counter* of the message.
            A. Clear the Message Reception State bitmap for the group session from the peer.
            B. Mark the peer 's group key data message counter as synchronized.
        ii. Process the message.
    c. If the key has a *cache-and-sync* security policy, the receiver SHALL:
        i. If MCSP is not in progress for the given peer Node ID and group key:
            A. Store the message for later processing.
            B. Proceed to Section 4.18.5, “Message Counter Synchronization Exchange”.
        ii. Otherwise, do not process the message.
            A. An implementation MAY queue the message for later processing after MCSP completes if resources allow.

For each peer Node ID and group key pair there SHALL be at most one synchronization exchange outstanding at a time.

4.18.5. Message Counter Synchronization Exchange

A message synchronization exchange starts by sending the MsgCounterSyncReq message to the peer Node that sent the message with unknown message counter. When a synchronization request is triggered by an incoming multicast message, the Node SHALL first wait for a uniformly random amount of time between 0 and MSG_COUNTER_SYNC_REQ_JITTER.

The sender of the *MsgCounterSyncReq* message SHALL:

1. Set the Message Header fields as follows:
    a. The S Flag SHALL be set to 1.
    b. The DSIZ field SHALL be set to 1.
    c. The P Flag SHALL be set to 1.
    d. The C Flag SHALL be set to 1.
    e. The Session Type field SHALL be set to 1.
    f. The Session ID field SHALL be set to the Group Session Id for the operational group key

being synchronized.

g. The Source Node ID field SHALL be set to the Node ID of the sender Node.

h. The Destination Node ID field SHALL be set to the Source Node ID of the message that triggered the synchronization attempt.

2. Create a new synchronization Exchange.
   a. The Exchange ID of the message SHALL be set to match the new Exchange.
   b. The I Flag SHALL be set to 1.
   c. The A Flag SHALL be set to 0.
   d. The R Flag SHALL be set to 1.
3. Set the *Challenge* field to the value returned by `Crypto_DRBG(len = 8 * 8)` and store that value to resolve synchronization responses from the destination peer.
4. Arm a timer to enforce that a synchronization response is received before MSG_COUNTER_SYNC_TIMEOUT.
   a. Upon firing of the timer:
      i. The synchronization exchange SHALL be closed.
      ii. Any message waiting on synchronization associated with the exchange SHALL be discarded.
   b. The timer SHALL be stopped upon receipt of an authenticated *MsgCounterSyncRsp* message that matches:
      i. The Source Node ID field matches the Destination Node ID field of the most recent *MsgCounterSyncReq* message generated for that Node.
      ii. The *Response* field corresponds to the *Challenge* field of the *MsgCounterSyncReq* message.
5. Invoke Section 4.7.1, “Message Transmission” using parameters from step 1 to encrypt and then send the request message over UDP to the IPv6 unicast address of the destination.
   a. The request message SHALL use the same operational group key as the message which triggered synchronization.
   b. The group key SHALL be known/derivable by both parties (sender and receiver).

The receiver of *MsgCounterSyncReq* SHALL:

1. Verify the Destination Node ID field SHALL match the Node ID of the receiver, otherwise discard the message.
2. Respond with *MsgCounterSyncRsp*.

The sender of the *MsgCounterSyncRsp* response message SHALL:

1. Set the Message Header fields as follows:
   a. The S Flag SHALL be set to 1.
   b. The DSIZ field SHALL be set to 1.

c. The P Flag SHALL be set to 1.

d. The C Flag SHALL be set to 1.

e. The Session Type field SHALL be set to 1.

f. The Session ID field SHALL be set to the Group Session Id for the operational group key being synchronized.

g. The Source Node ID field SHALL be set to the Node ID of the sender Node.

h. The Destination Node ID field SHALL be set to the Source Node ID of the *MsgCounterSyncReq*.

2. Set the MsgCounterSyncRsp payload fields as follows:

a. The *Response* field SHALL be set to the value of the *Challenge* field from the *MsgCounterSyncReq*.

b. The *Synchronized Counter* field SHALL be set to the current Global Group Encrypted Data Message Counter of the sender.

3. Use the same exchange context as the *MsgCounterSyncReq* being responded to.

a. The Exchange ID of the message SHALL be set to the *Exchange ID* of the *MsgCounterSyncReq*.

b. The I Flag SHALL be set to 0.

c. The A Flag SHALL be set to 1.

d. The R Flag SHALL be set to 1.

4. Invoke Section 4.7.1, “Message Transmission” using parameters from step 1 to encrypt and then send the response message over UDP to the IPv6 unicast address of the destination.

The receiver of the *MsgCounterSyncRsp* message SHALL:

1. Verify the *MsgCounterSyncRsp* matches a previously sent *MsgCounterSyncReq*:

a. An active synchronization exchange SHALL exist with the source node.

b. The *Exchange ID* field SHALL match the *Exchange ID* used for the original *MsgCounterSyncReq* message.

c. The *Response* field SHALL match the *Challenge* field used for the original *MsgCounterSyncReq* message.

d. The Destination Node ID field SHALL match the Source Node ID of the original *MsgCounterSyncReq* message.

e. The Source Node ID field SHALL match the Destination Node ID of the original *MsgCounterSyncReq* message.

2. On verification failure:

a. Silently ignore the *MsgCounterSyncRsp* message.

3. On verification success:

a. Set the peer’s group key data message counter to *Synchronized Counter*.

b. Clear the Section 4.6.5.1, “Message Reception State” bitmap for the peer.

c. Mark the peer 's group key data message counter as synchronized.

d. Resume processing of any queued message that triggered synchronization according to Section 4.6.7, “Counter Processing of Incoming Messages”.

   i. If more than one message is queued from the synchronized peer, using the same operational group key, the messages SHALL be processed in the order received.

e. Close the synchronization exchange created for the original *MsgCounterSyncReq* message.

4.18.6. Message Counter Synchronization Session Context

While conducting Message Counter Synchronization with a peer, nodes SHALL maintain the following session context. For nodes initiating message counter synchronization, this context SHALL be maintained throughout the full exchange of *MsgCounterSyncReq* and *MsgCounterSyncRsp* messages. For nodes responding to *MsgCounterSyncReq* messages, the context SHALL only be maintained long enough to generate and successfully transmit the *MsgCounterSyncRsp* message.

1. `Fabric Index`: Records the Index for the Fabric within which nodes are conducting message counter synchronization.
   - Fabric Index is derived by identification of an Operational Group Key associated with the fabric through successful decryption with that key and verification of the Message Integrity Check. For nodes initiating counter synchronization, this occurs at decryption of an inbound groupcast message. For nodes in the responder role, this occurs at decryption of an inbound *MsgCounterSyncReq* message.
2. `Peer Node ID`: Records the node ID of the peer with which message counter synchronization is being conducted.
   - For nodes initiating message counter synchronization, this is the node ID of the responder. For nodes responding to message counter synchronization, this is the node ID of the initiator.
3. `Role`: Records whether the node is the initiator of or responder to message counter synchronization.
   - Together, *Fabric Index*, *Peer Node ID* and *Role* comprise a unique key that can be used to match incoming messages to ongoing MCSP exchanges.
4. `Message Reception State`: Provides tracking for the Control Message Counter of the remote peer.
5. `Peer IP Address`: The unicast IP address of the peer.
6. `Peer Port`: The receiving port of the peer.
7. `Operational Group Key`: The Operational Group Key that is being used to encrypt messages within the counter synchronization exchange.
8. `Group Session ID`: Records the Group Session ID derived from the Operational Group Key that is being used to encrypt messages within the counter synchronization exchange.

4.18.7. Sequence Diagram

The following sequence diagram shows an example of how message counter synchronization behaves in the most common scenario.

4.18.7.1. Scenario 1 — Multicast Receiver Initiated

Assumptions:

- *Sender* transmits a multicast message.
- *Receiver* does not know *Sender* 's message counter.

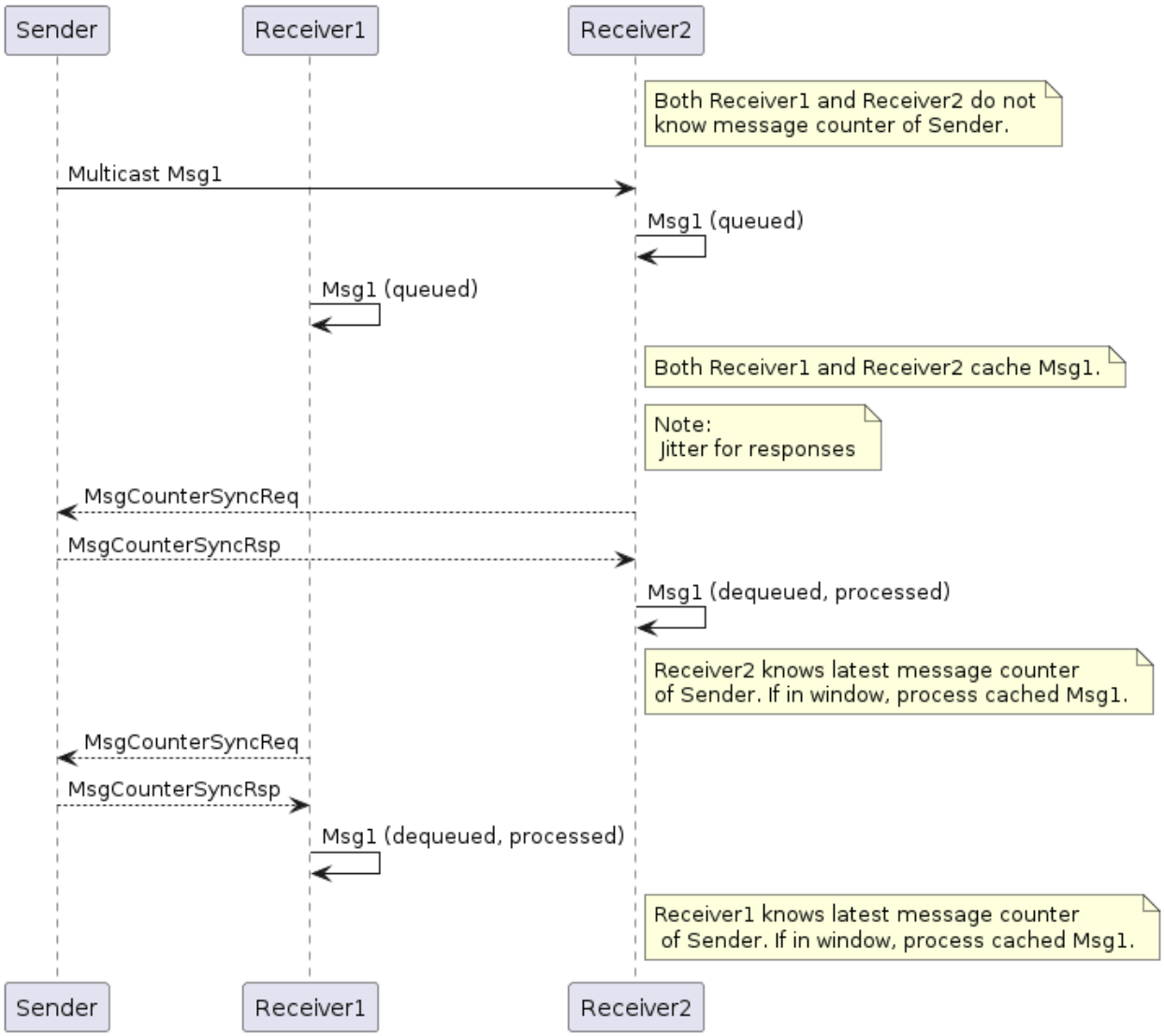

Figure 21. Multicast Receiver Initiated Message Counter Synchronization

Sequence:

- *Sender*:
  - Generates, encrypts and sends *Msg1* as a multicast message. *Msg1* could be:
    - Regular message that starts encrypted group communication with receivers *Receiver1* and *Receiver2*.
- Receivers *Receiver1* and *Receiver2* each:
  - Receive, decrypt, authenticate, and cache *Msg1* message for later processing.
    - Generate, encrypt, and send *MsgCounterSyncReq* message.

